



UNIVERSITY OF TARTU

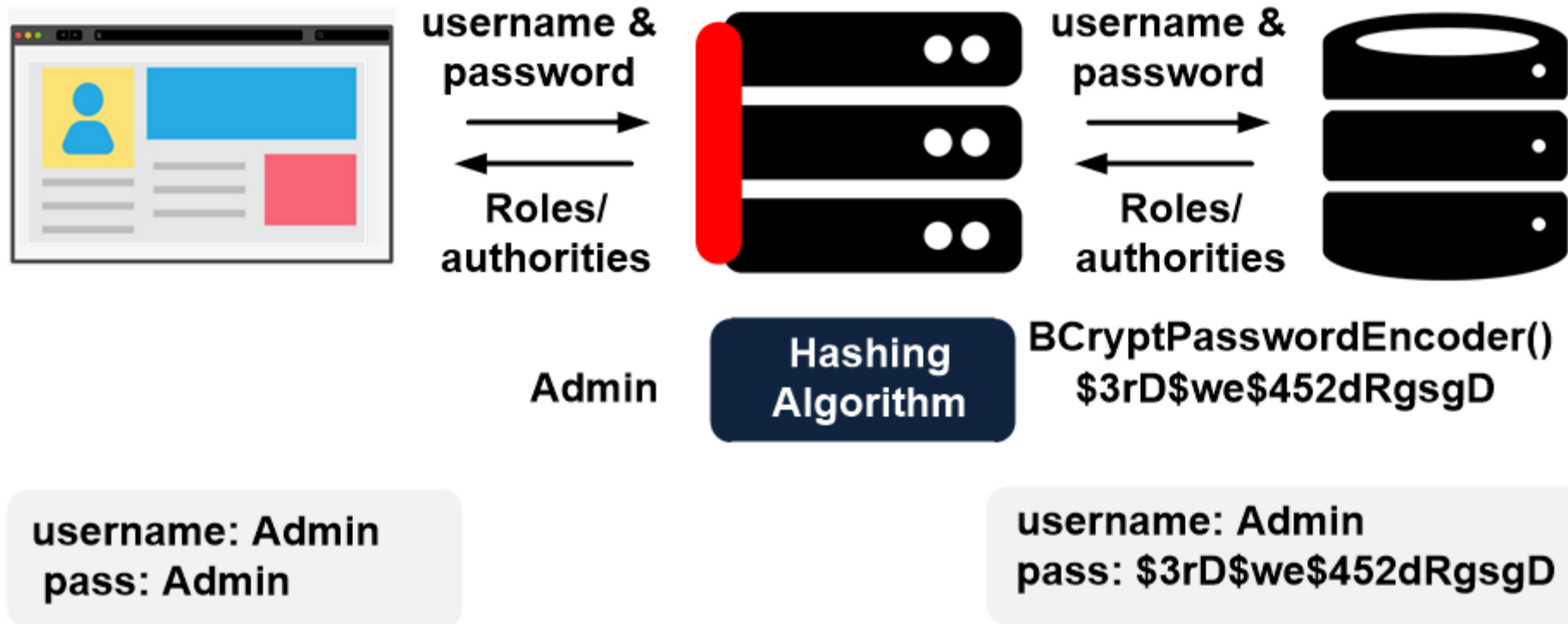


Enterprise System Integration (MTAT.03.229)

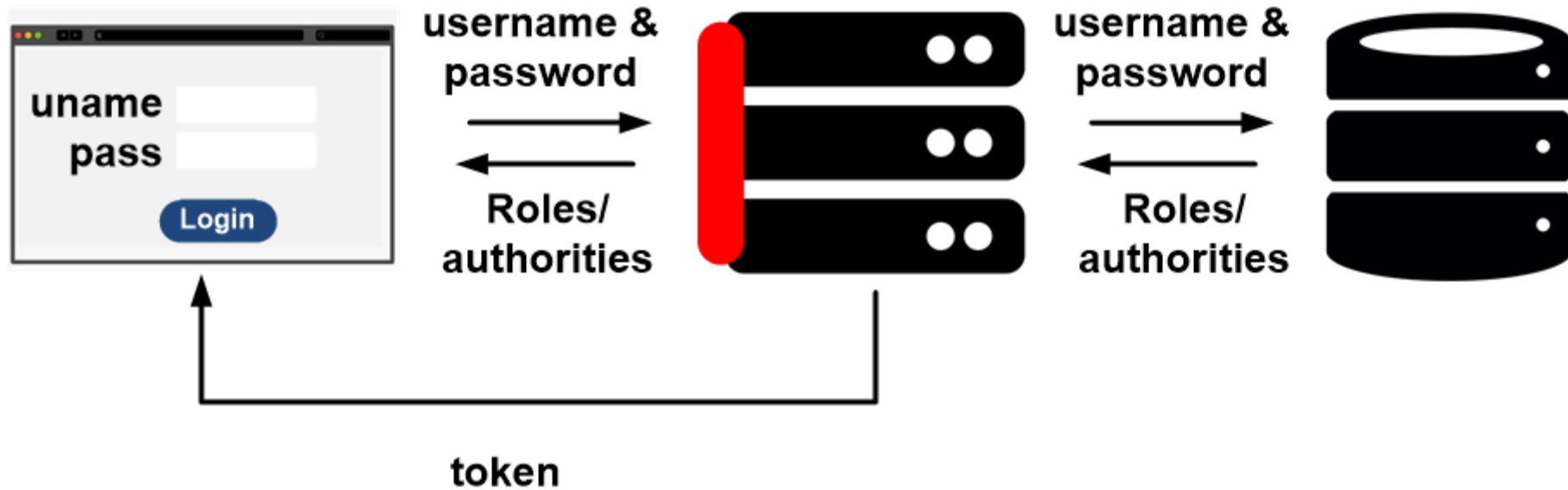
LECTURE 10: SECURITY ASPECTS OF INTEGRATION – II

**MOHAMAD GHARIB
UNIVERSITY OF TARTU**

What is wrong with this system? - recap



What is wrong with this system? - recap



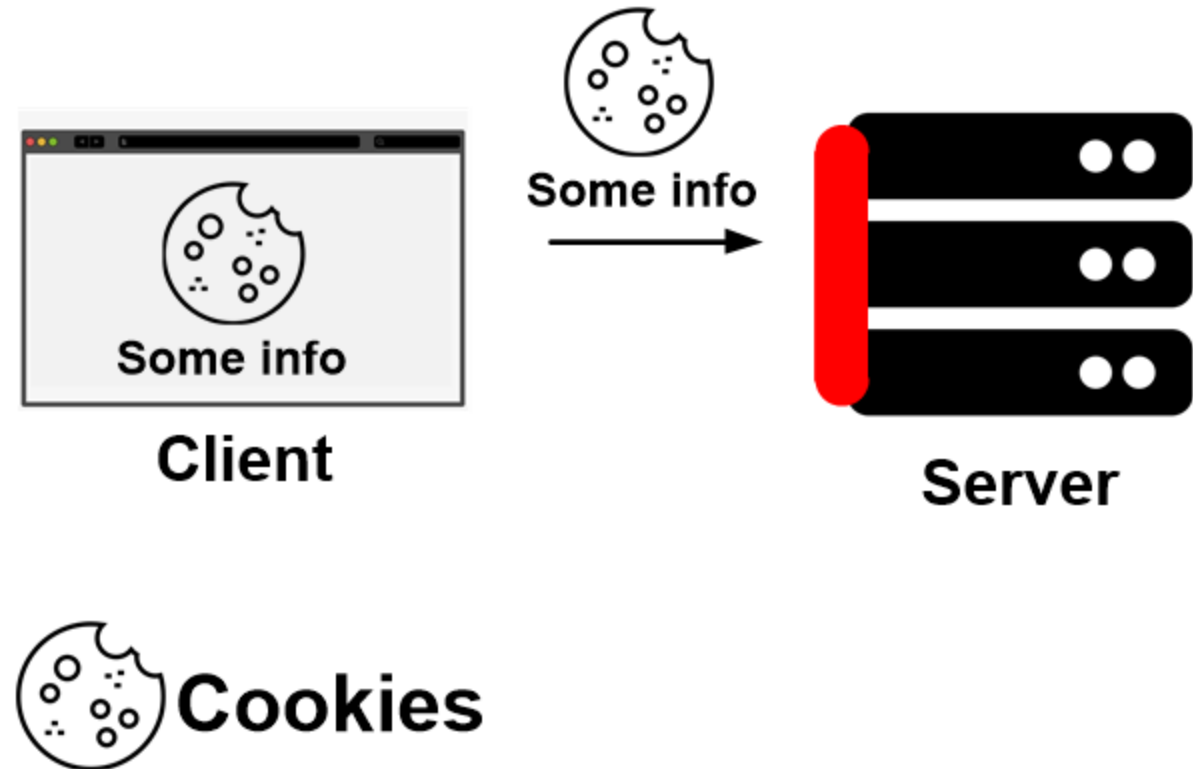
Cookies - recap

A cookie is created when you visit a website.

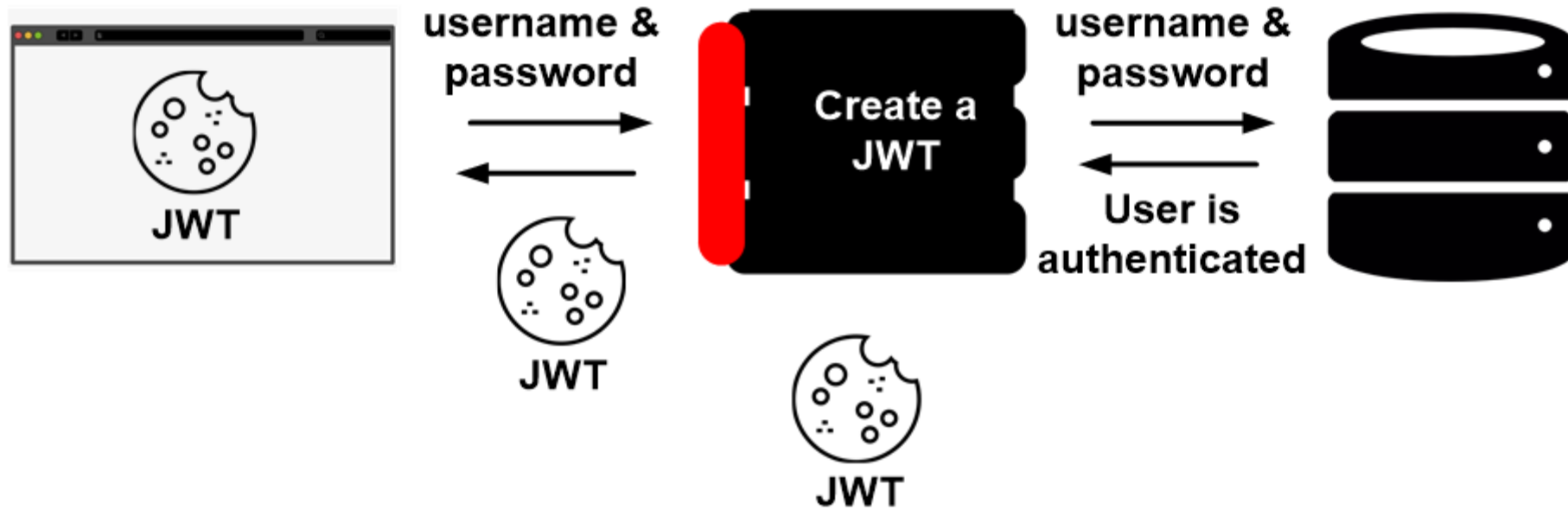
A **cookie** can be “session” or (non-persistent **cookie**), or persistent **cookie**.

After its creation, it is sent to be stored in the user browser.

When you revisit the website, your browser sends the information stored in the cookie **back** to the site.



How JWT is created, signed and verified



After creating the JWT, it is sent to be stored in the user's browser.

How JWT is created, signed and verified



Result is OK proceed to resource
Result is not OK, redirect to login/signup page

After creating the JWT, it is sent to be stored in the user's browser.

When the user **revisit** the website, the browser automatically sends the JWT (and any cookies) to the **server**.

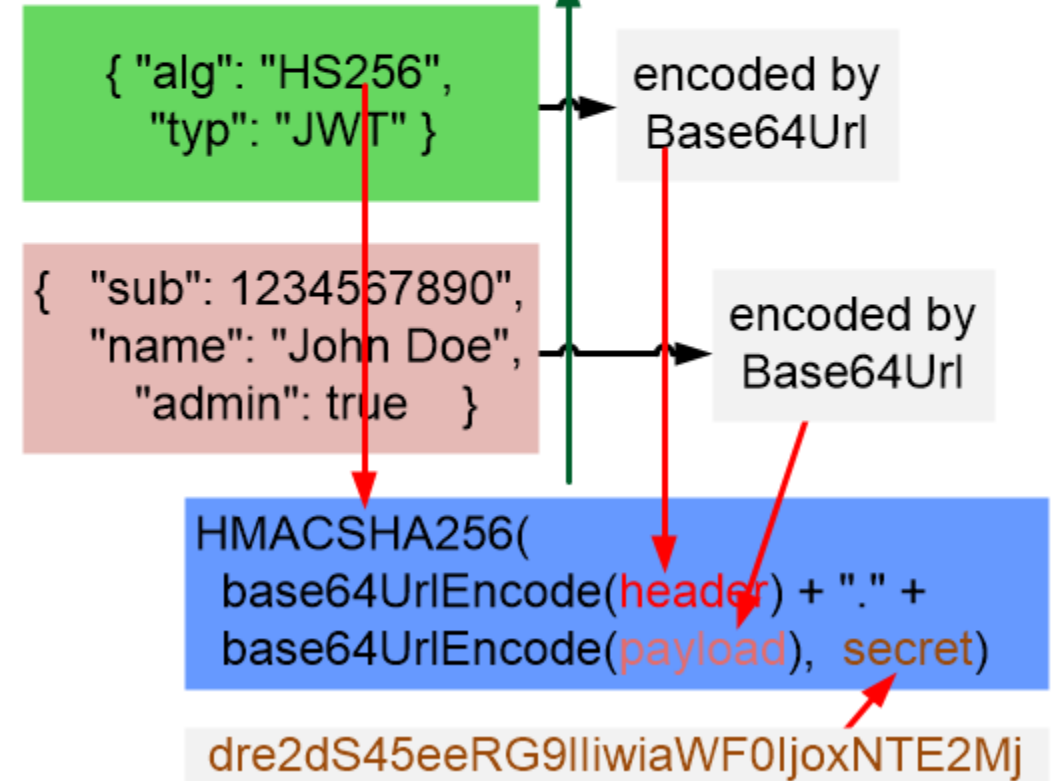
JWT structure - recap



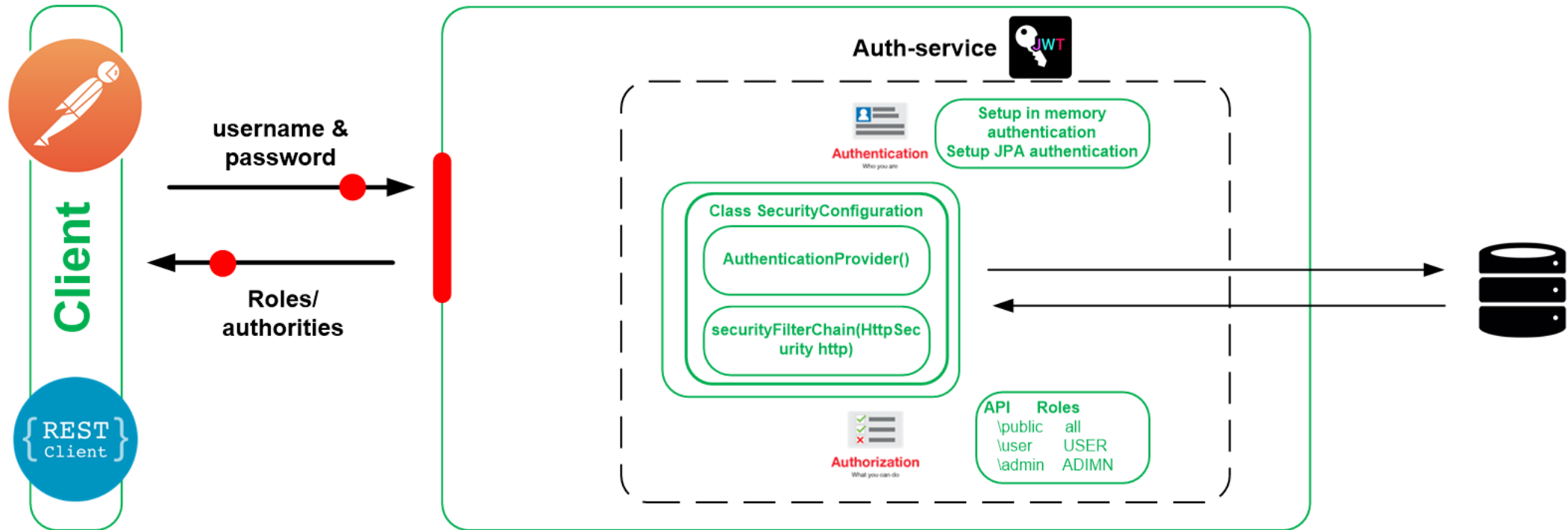
The signature is used to verify that the message was not changed, and, in the case of tokens signed with a private key, it can also verify that the sender of the JWT is who it says it is.

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MzE2MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

Header	Payload	Signature
Header	consists of two parts: the type of the token, which is JWT, and the signing algorithm being used, such as HMAC SHA256 or RSA.	
Payload	contains the claims, which are statements about the user and additional data.	
Signature	makes the token secure by taking the encoded header, the encoded payload, a secret, the algorithm specified in the header, and sign that.	



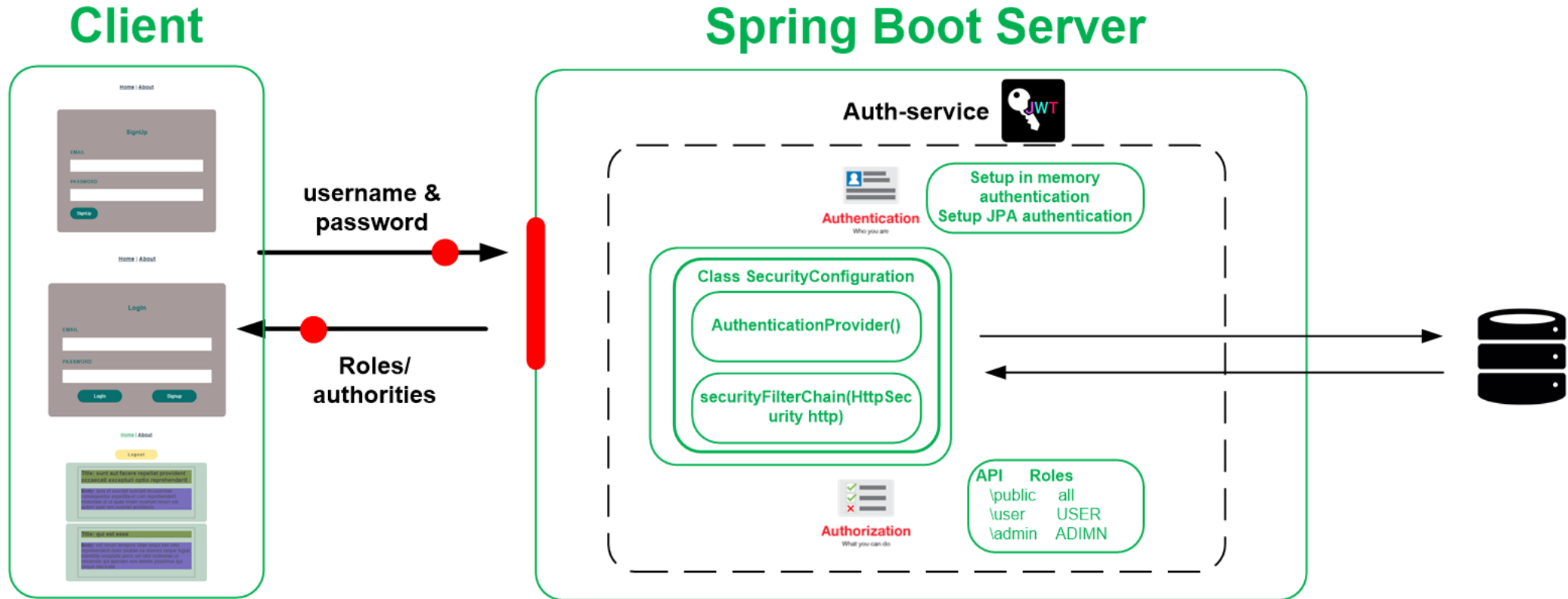
Auth Service for a Spring Boot App - recap



Building a “secure” project using JWT

backend + frontend

A "Secure" project (SB + VueJs)





Building a “secure” App - JWT

Our App includes:

A front-end (Vue.js App) offers:

- A home page (protected), only authenticated users can reach.
- Login and signup pages.
- A logout button.

A backend (Spring Boot App) allows:
(1) Register new users (set roles); (2) Login registered users; and (3) Verify authenticated users.

A database (Postgres) contains the user’s data (id, name, password, role).

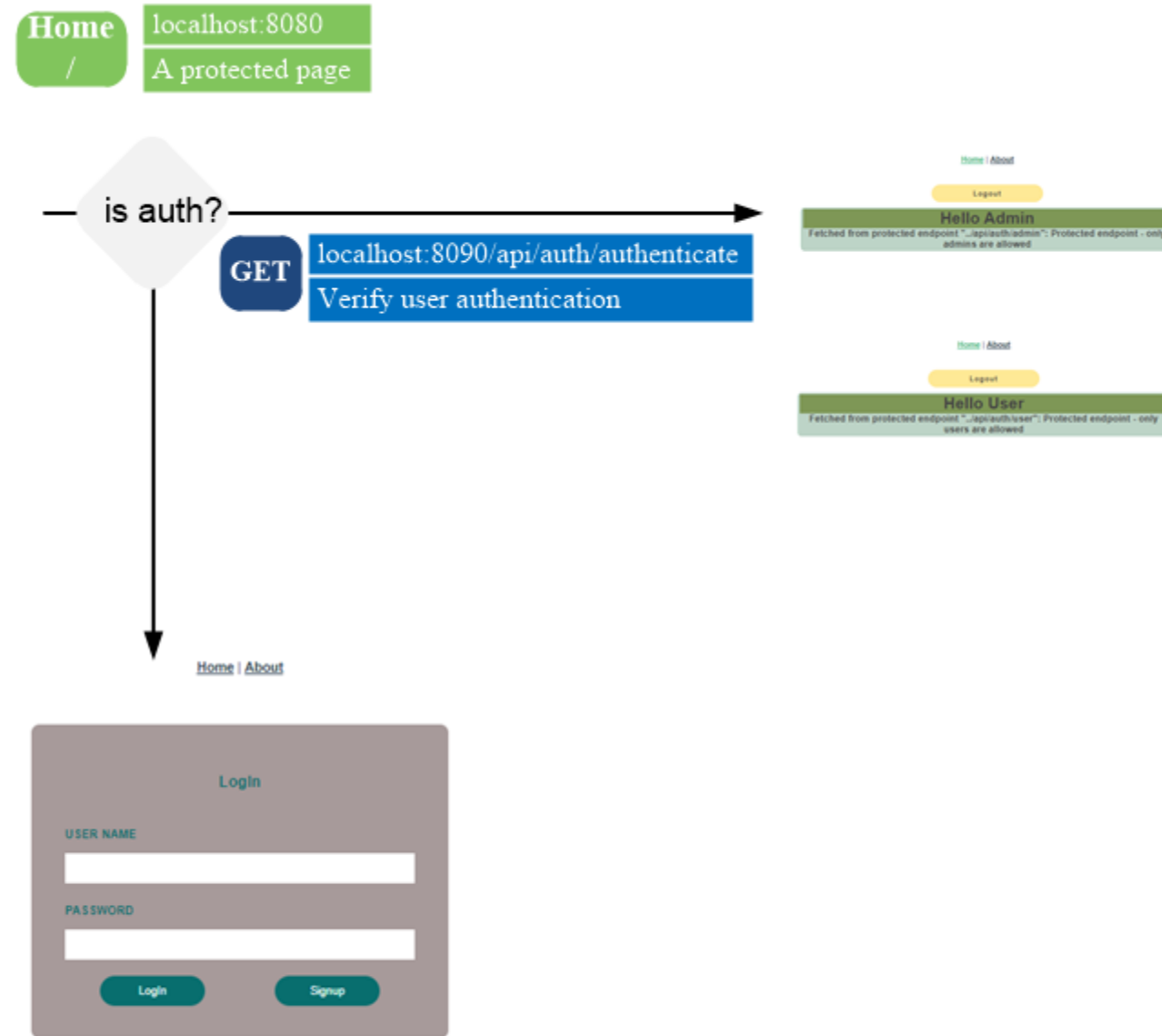
Method	URI/PATH	Actions
POST	/api/auth/signup	Create/register a new user
POST	/api/auth/login	Login a registered user
GET	/api/auth/authenticate	Authenticate a user
GET	/api/auth/public	Public endpoint
GET	/api/auth/admin	Protected endpoint - only users (Role ADMIN) are allowed
GET	/api/auth/user	Protected endpoint - only users (Role USER) are allowed



Building a “secure” App - JWT

If a user tries to reach the homepage, there is a “navigation guard” that will check whether a user is authenticated.

How?





Building a “secure” App - JWT

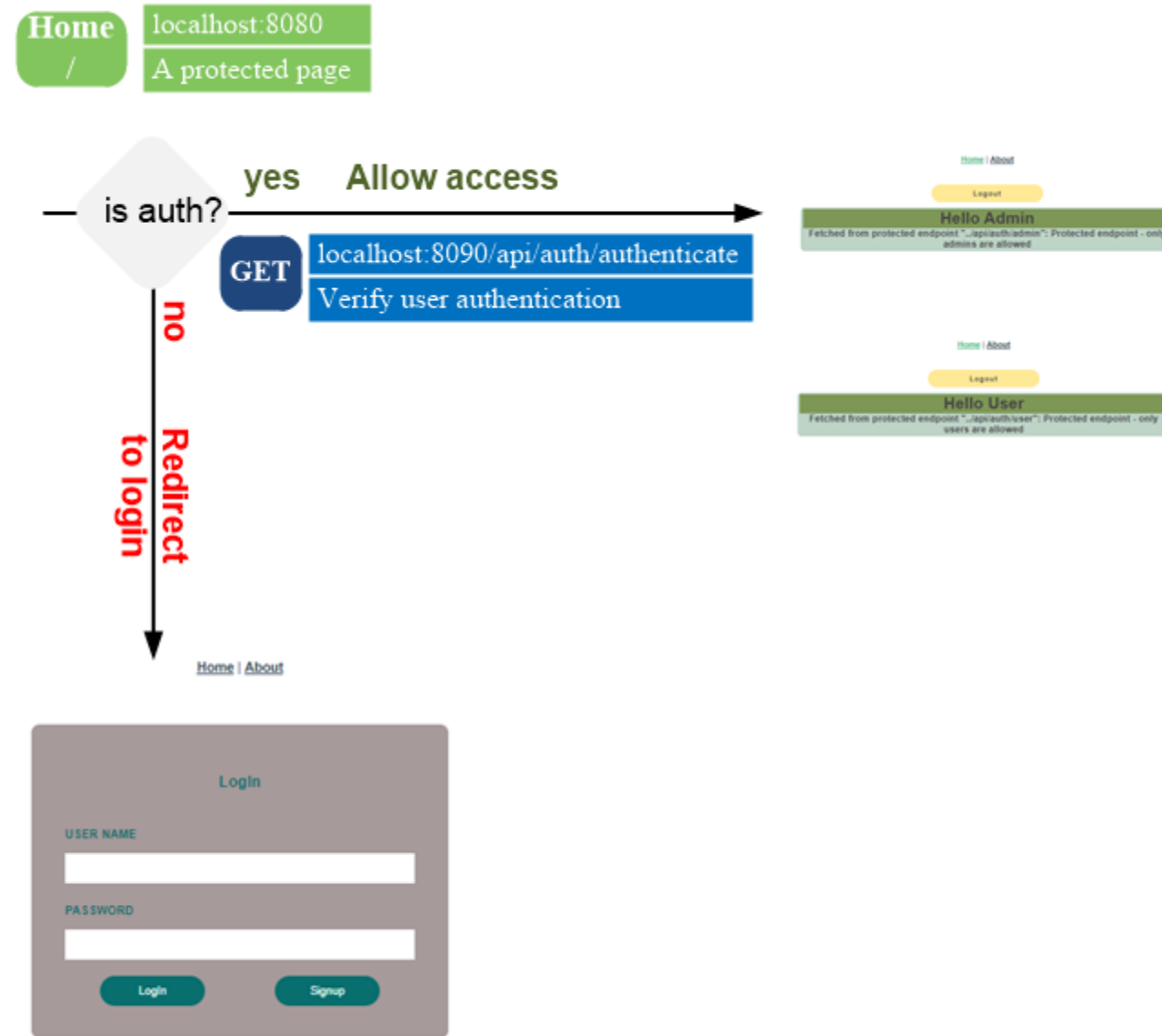
If a user tries to reach the homepage, there is a “navigation guard” that will check whether a user is authenticated.

How?

Sends a **GET request** to the server for **verifying the jwt**, if it exist.

The server will **respond** by **true** or **false** concerning the **authentication** of the user.

If the user is **authenticated** (there is a **valid jwt**), she will reach the homepage. Otherwise, she will be **redirected** to the **login page**.



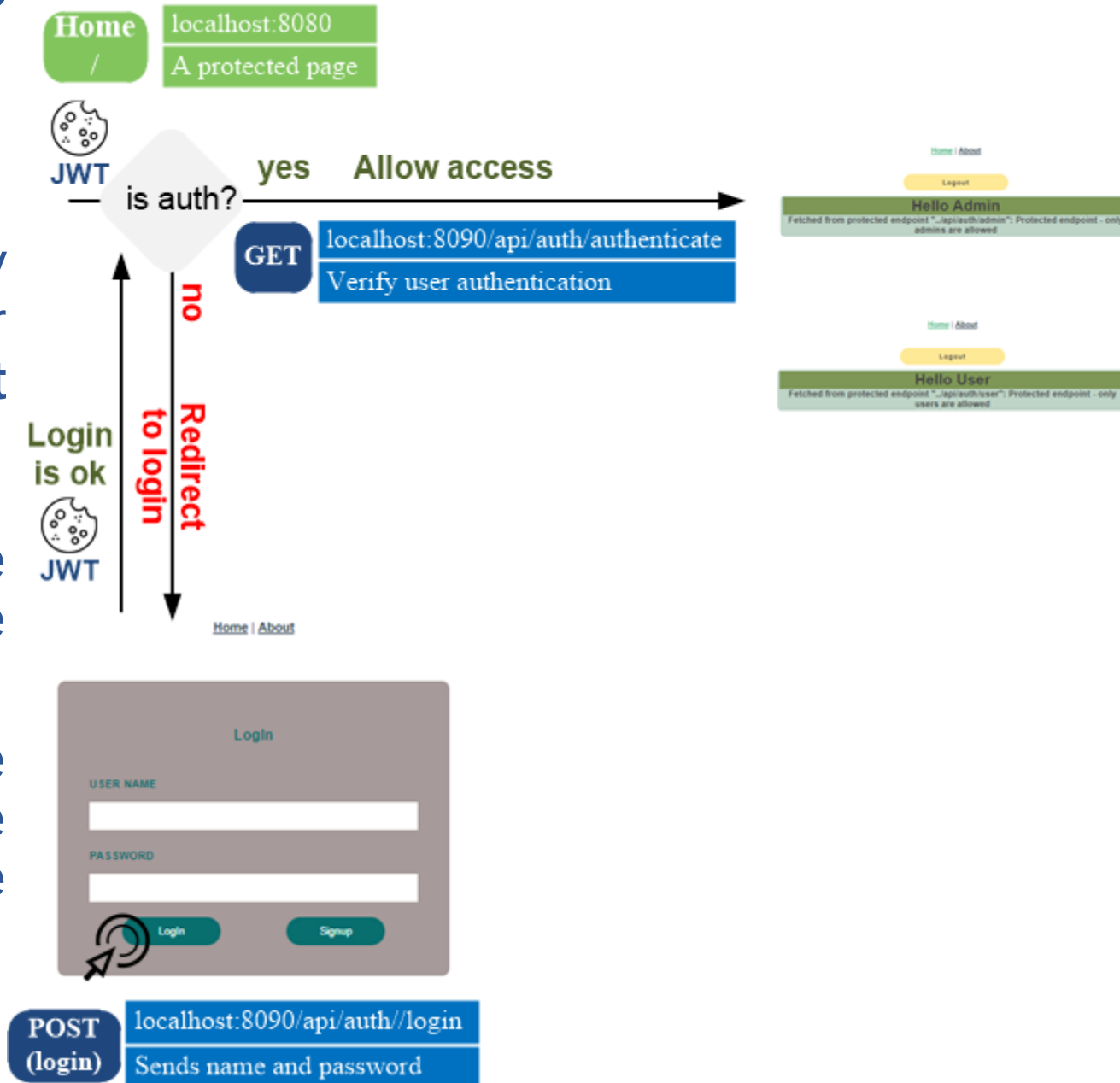


Building a “secure” App - JWT

If the user is **already registered**, they can use the **login page** to send her **credentials** through a **POST request** to the server.

If the **credentials** are **correct**, she will be **logged in** and a **jwt** token will be created and send back to her browser.

This will allow the user to pass the authentication check offered by the “**navigation guard**”, and reach the **protected homepage**.



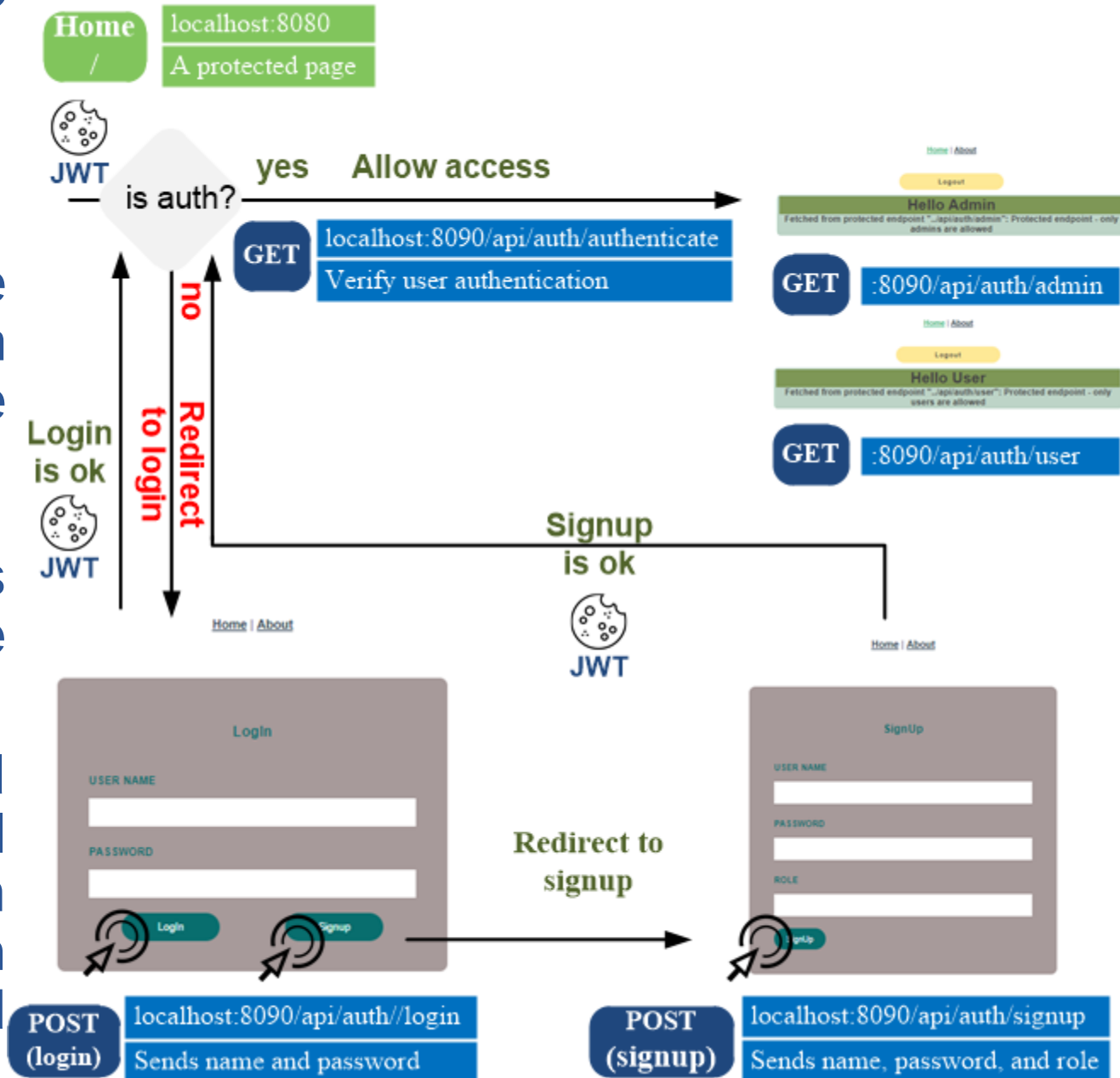


Building a “secure” App - JWT

If the user is not already **signed up**, she will not be able to **login in**, but she can navigate to the **signup page** from the **login page**.

The user can **signup** by sending its credentials through the **signup page** as a **POST request** to the server.

This will create a **jwt** token and send back to the user browser, which will allow her to pass the authentication check offered by the “**navigation guard**”, and reach the protected **homepage**.

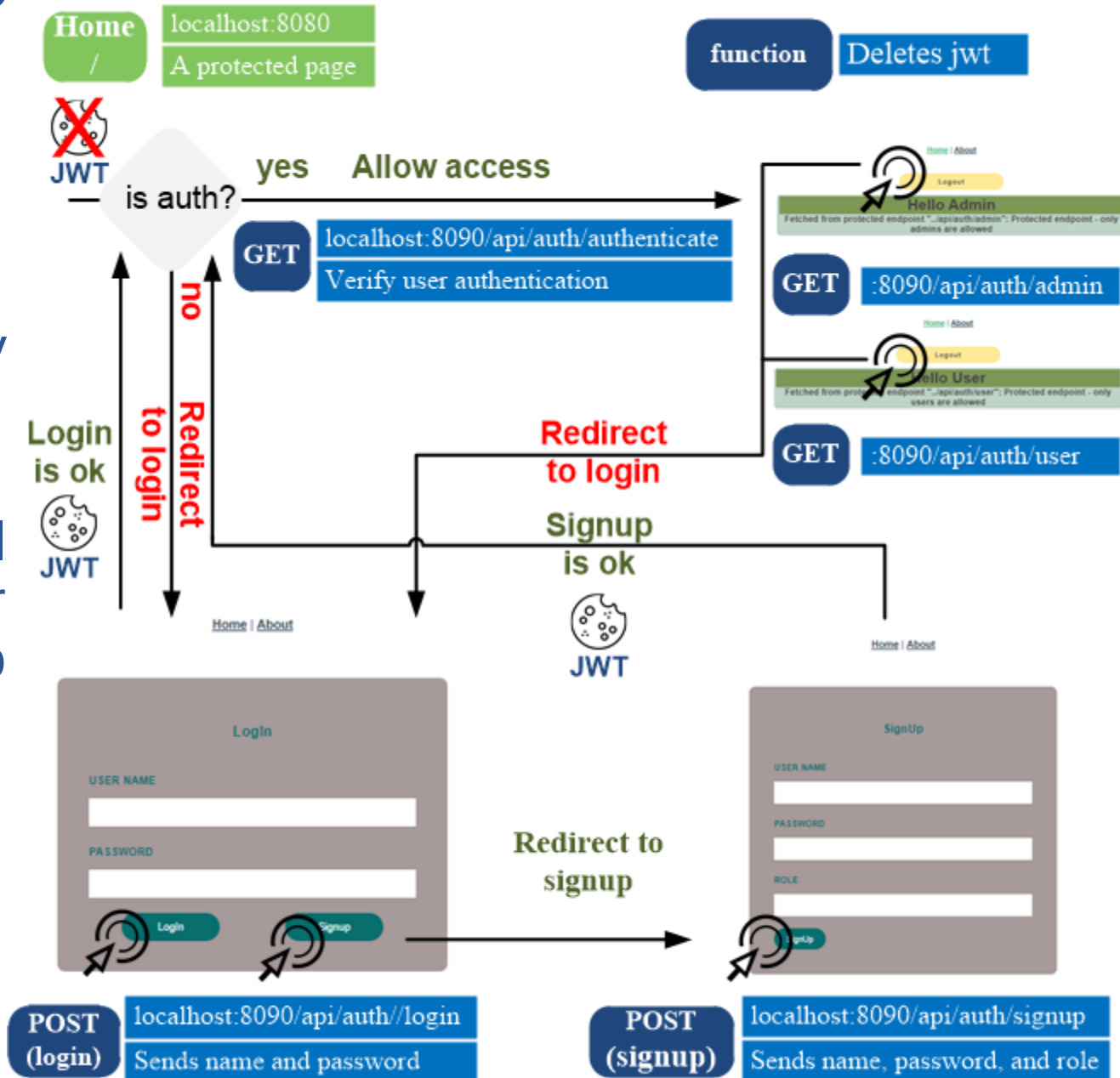




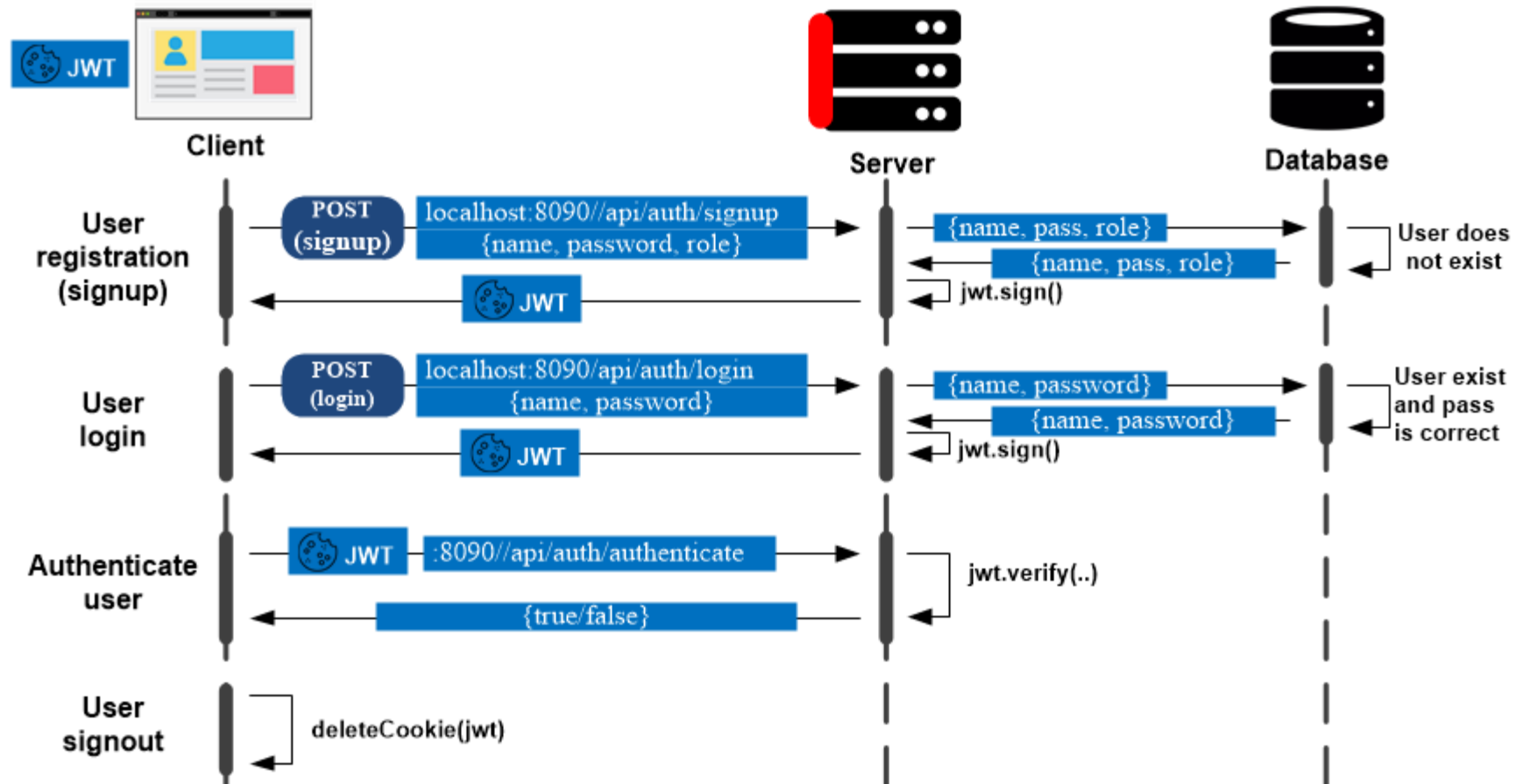
Building a “secure” App - JWT

A logged in user, can log out by pressing on the **logout** button.

This will **deletes** the token from the **local storage**, where it is saved. After logging out, the user is **redirected** to the **login** page.



Building a “secure” App - JWT



Building a “secure” App - Backend

Auth-service – Backend

```
// UserController.java
...
@GetMapping("/authenticate")
public Boolean authenticate(@RequestHeader("Authorization") String header) {
    String token = header.replace("Bearer ", "");
    log.info(" authenticate - token {} ", token);
    return jwtService.validateToken(token);
}
...
```

Auth-service – Backend

```
// UserController.java
```

```
...
@GetMapping("/authenticate")
public Boolean authenticate(@RequestHeader("Authorization") String header) {
    String token = header.replace("Bearer ", "");
    log.info(" authenticate - token {} ", token);
    return jwtService.validateToken(token);
}
...
```

```
// JwtService.java
```

```
...
public boolean validateToken(String token) {
    try { // verify the integrity of the JWT token
        Jwts.parserBuilder().setSigningKey(signingKey()).build().parseClaimsJws(token);
        return true;
    } catch (JwtException | IllegalArgumentException e) {
        // JwtException can be thrown if the token is invalid or has expired
        return false;
    }
}
...
```

Building a “secure” App - Frontend

Navigation guard

```
// ../router/index.js

...

import auth from "../auth";

const routes = [{
  path: "/",
  name: "home",
  component: HomeView,
  beforeEnter: async(to, from, next) => {
    let authResult = await auth.authenticated();
    if (!authResult) {
      next('/login')
    } else {
      next();
    }
  }
},
...

```

Auth.js

```
// ../auth.js

import jwt_decode from 'jwt-decode';

export default {
  user: { authenticated: false,
    roles: '',
    username: ''},
  authenticated: async function() {
    const token = localStorage.getItem('jwtToken');
    // if there is a token, get the user roles, and try to authenticate
    if (token) {
      this.user.roles = jwt_decode(token).roles;
      // Set the Authorization header with the token value
      const headers = {
        'Content-Type': 'application/json',
        Authorization: `Bearer ${token}`
      };
    }
  }
};
```

Auth.js

```
// ../auth.js

import jwt_decode from 'jwt-decode';

export // ../auth.js

    await fetch("http://localhost:8090/api/auth/authenticate", { headers })
        .then((response) => response.json())
        .then((data) => {
            this.user.authenticated = data;
            console.log(data);
        })
        .catch(err => console.log(err.message));
    return this.user.authenticated; //returns a Boolean value
} else
    // there is no token, we cannot authenticate
    return false;

},
```


Auth.js

```
// ../auth.js
```

```
import jwt_decode from 'jwt-decode';
```

```
export // ../auth.js
```

```
await // ../auth.js
```

```
    hasARoleOf: function(roles) {  
        if (this.user.roles)  
            return this.user.roles == roles;  
    },  
  
    logout: function() {  
        localStorage.removeItem('jwtToken');  
        this.user = { roles: "", username: "", authenticated: false };  
    },  
}
```

Login.vue - Template

```
// login.vue

<template>
  <div class="form">
    <h3>LogIn</h3>
    <label for="username">User Name</label>
    <input type="text" name="username" required v-model="username">
    <label for="password">Password</label>
    <input type="password" name="password" required v-model="password">
    <div class="container">
      <button @click="LogIn" class="center">LogIn</button>
      <button @click='this.$router.push("/signup")' class="center">Signup</button>
    </div>
  </div>
</template>
```

Login.vue - LogIn()

```
// login.vue

sync LogIn() {var data = {name: this.username,
                        password: this.password};
  await fetch("http://localhost:8090/api/auth/login", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify(data)})
  .then(response => response.text())
  .then(response => {this.token = response;
    if (this.token.startsWith("eyJ"))
    {   this.decodedToken = jwt_decode(this.token);
        this.roles = this.decodedToken.roles
        localStorage.setItem('jwtToken', this.token);
        this.$router.push("/");}
  }) .catch((e) => {console.log(e);
    console.log("error");});},},},}
```

SignUp.vue - Template

```
// SignUp.vue

<template>
  <div class="form">
    <h3>SignUp</h3>
    <label for="username">User Name</label>
    <input type="text" name="username" required v-model="username">
    <label for="password">Password</label>
    <input type="password" name="password" required v-model="password">
    <label for="roles">Role</label>
    <input type="text" name="roles" required v-model="roles">
    <button @click="SignUp" class="SignUp">SignUp</button>
  </div>
</template>
```

SignUp.vue - SignUp()

```
// SignUp.vue
```

```
async SignUp() {  
  var data = {name: this.username, password: this.password, roles: this.roles};  
  await fetch("http://localhost:8090/api/auth/signup", {  
    method: "POST",  
    headers: {"Content-Type": "application/json"},  
    body: JSON.stringify(data),    })  
  .then(response => response.text())  
  .then(response => {  
    this.token = response;  
    if (this.token.startsWith("eyJ")) {  
      this.decodedToken = jwt_decode(this.token);  
      this.roles = this.decodedToken.roles  
      localStorage.setItem('jwtToken', this.token);  
      console.log(localStorage.getItem('jwtToken'));  
      this.$router.push("/");}})  
  .catch((e) => {console.log(e); console.log("error");});},},  
}
```

HomeView.vue

```
// HomeView.vue
<template>
<div class="header">
<div>
  <button v-if = "authResult" @click="Logout" class="center">Logout</button>
  <div class="name" v-if="checkRoles('ADMIN')">
    <h1> Hello Admin</h1>
    <h3>  Fetched from protected endpoint "../api/auth/admin":{{AdminViewContent}}</h3>
  </div>
  <div class="name" v-if="checkRoles('USER')">
    <h1> Hello User</h1>
    <h3>  Fetched from protected endpoint "../api/auth/user": {{UserViewContent}} </h3>
  </div>
</div>
</div>
</template>
```



```
<template> // HomeView.vue
<div>
  ...
</div>
mounted() {
  if (this.checkRoles('ADMIN')) {
    const token = localStorage.getItem('jwtToken');
    // Set the Authorization header with the token value
    const headers = {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${token}`
    };
    fetch("http://localhost:8090/api/auth/admin", { headers })
      .then(response => response.text())
      .then(body => {
        this.AdminViewContent = body;
      })
      .catch(err => console.log(err.message))
  }
}
```



```
<template>
  // HomeView.vue
```

```
<div>...</div>  
<div>...</div>  
...  
// HomeView.vue
```

```

i ...
mounted() {
  ...
  if (this.checkRoles('USER')) {
    const token = localStorage.getItem('jwtToken');
    // Set the Authorization header with the token value
    const headers = {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${token}`
    };
    fetch("http://localhost:8090/api/auth/user", { headers })
      .then(response => response.text())
      .then(body => {
        this.UserViewContent = body;
      })
      .catch(err => console.log(err.message))
  }
}

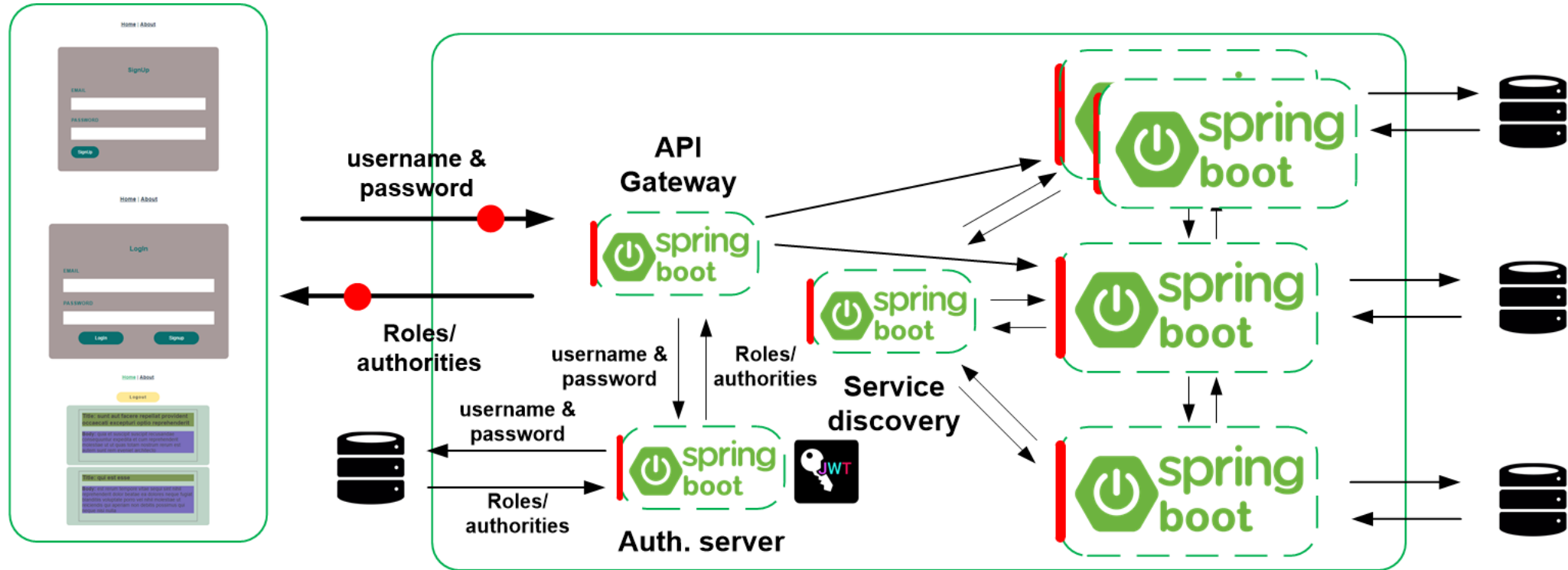
```


Building a “secure” Microservice-based project using JWT

backend + frontend

Building a “secure” M-based project

Client





Building a “secure” M-based project

Our App includes:

A front-end (Vue.js App) offers:

- A home page (protected), only authenticated users can reach.
- Login and signup pages.
- A logout button.

A backend (Spring Boot App) allows:
(1) Register new users (set roles); (2) Login registered users; and (3) Verify authenticated users.

A database (Postgres) contains the user’s data (id, name, password, role).

Method	URI/PATH	Actions
POST	:8080/api/auth/signup	Create/register a new user
POST	8080/api/auth/login	Login a registered user
GET	:8080/api/auth/authenticate	Authenticate a user
GET	:8080/api/products	Fetch and present all products for authenticated users



Building a “secure” M-based project

Our App includes:

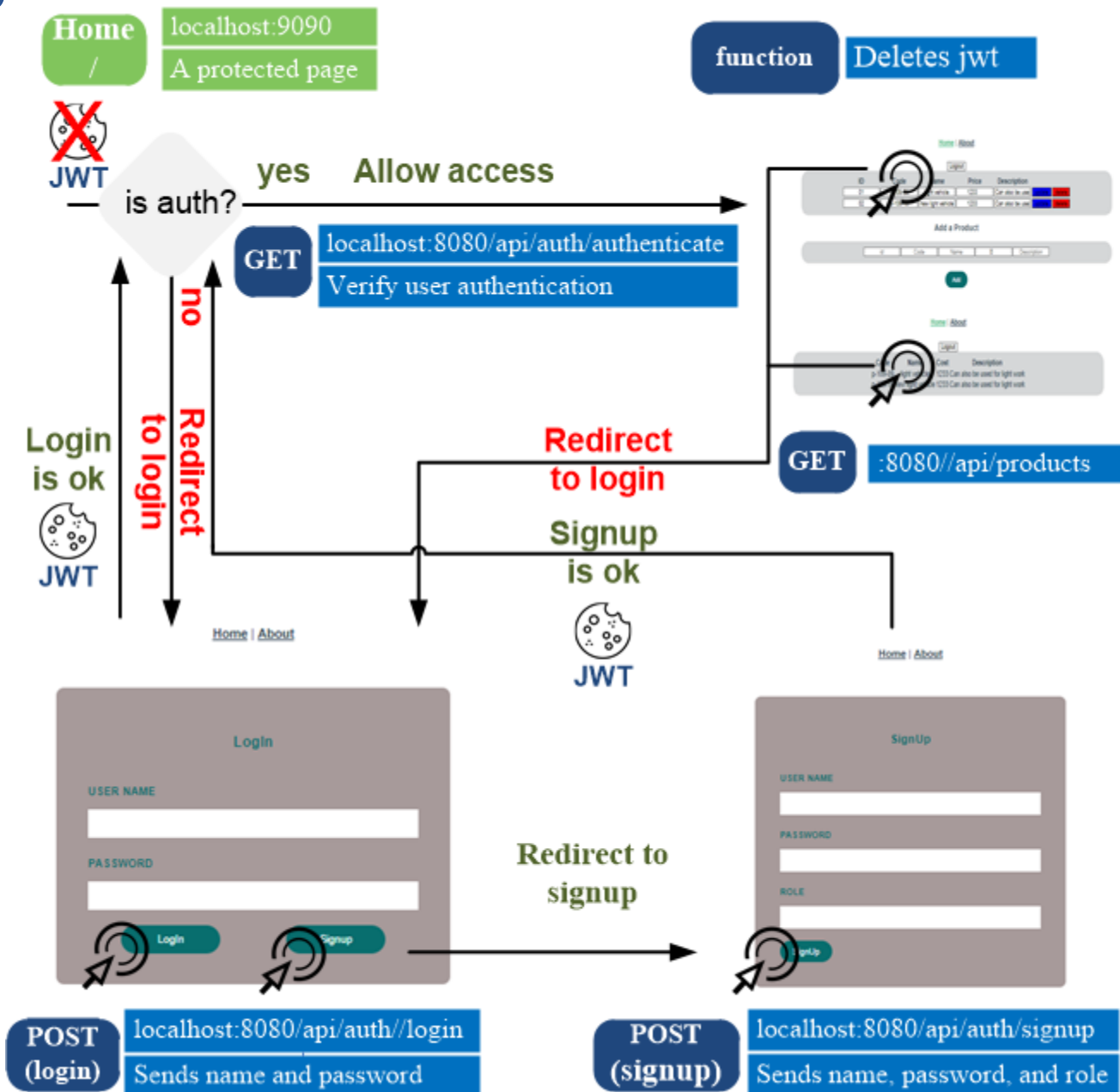
A front-end (Vue.js App) offers:

- A home page (protected), only authenticated users can reach.
- Login and signup pages.
- A logout button.

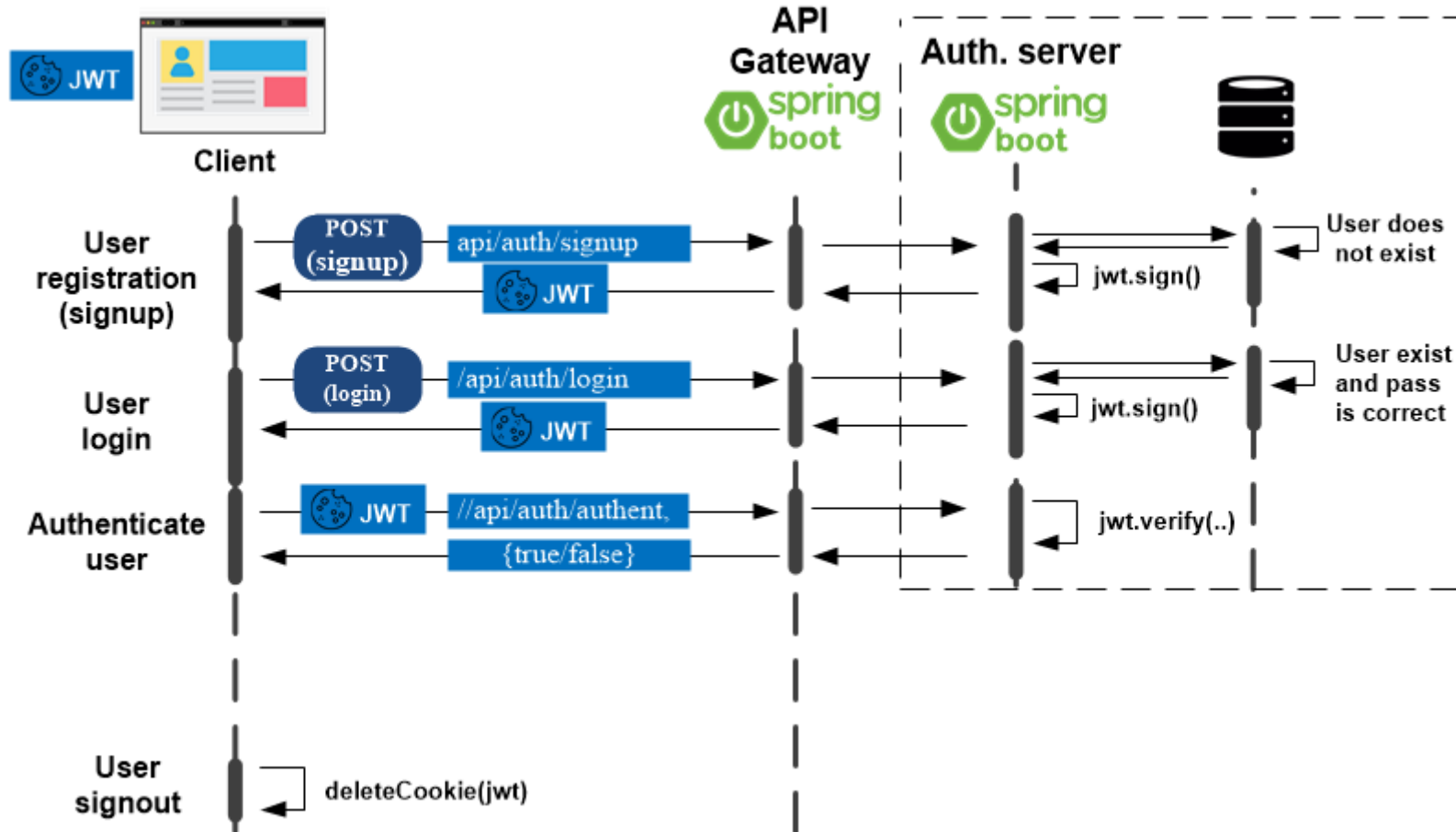
A backend (Spring Boot App) allows:

- (1) Register new users (set roles);
- (2) Login registered users; and
- (3) Verify authenticated users.

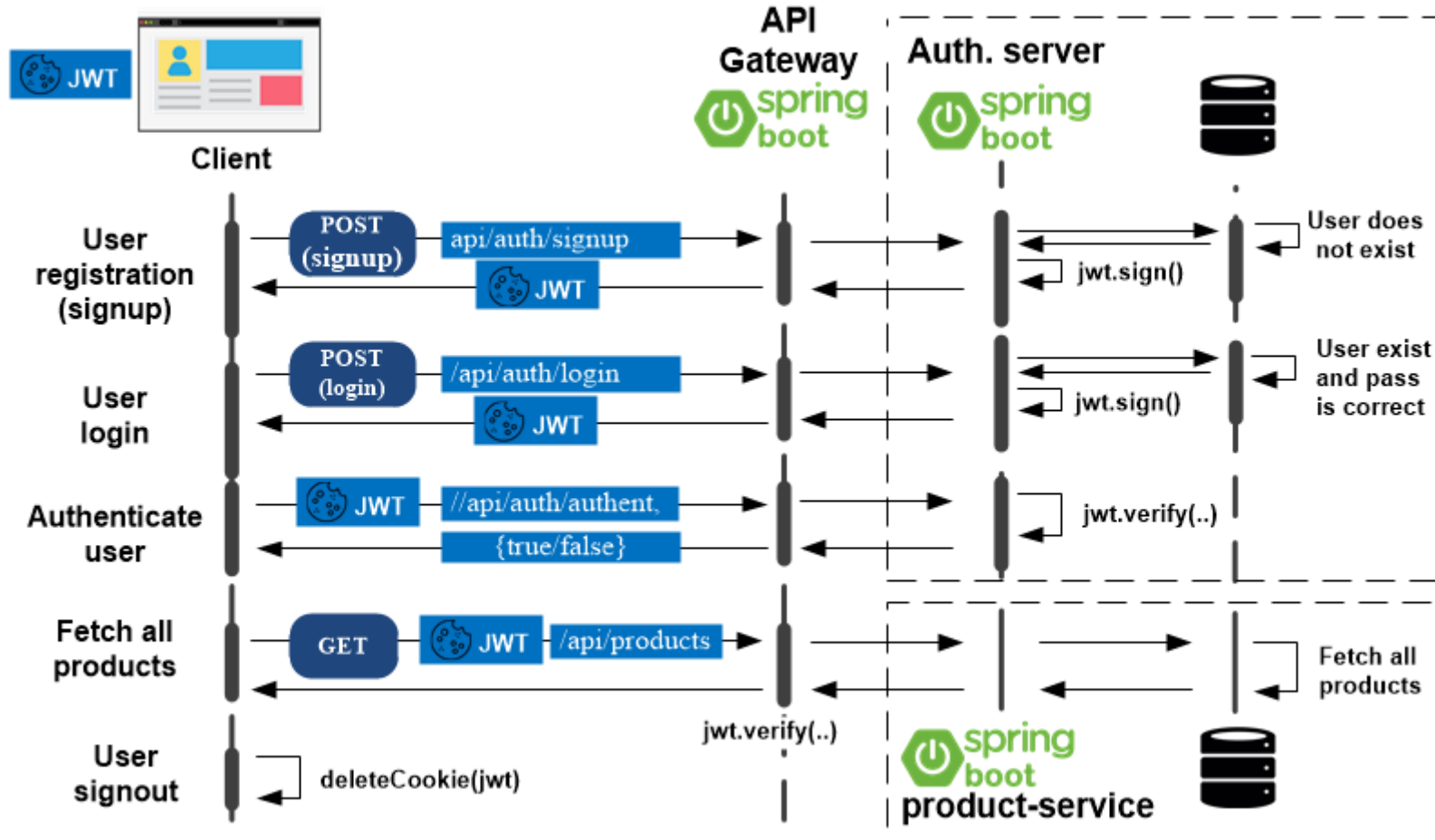
A database (Postgres) contains the user's data (id, name, password, role).



Building a “secure” M-based project



Building a “secure” M-based project



Building a “secure” M-based project

backend

api-gateway service

```
// filter/AuthenticationFilter.java
...
@Override
public GatewayFilter apply(Config config) {
    return ((exchange, chain) -> {
        // validator contains openApiEndpoints, a list of open endpoints
        if (validator.isSecured.test(exchange.getRequest())) {
            if (!exchange.getRequest().getHeaders().containsKey(HttpHeaders.AUTHORIZATION)) {
                throw new RuntimeException("missing authorization header");}
            String authHeader =
                exchange.getRequest().getHeaders().get(HttpHeaders.AUTHORIZATION).get(0);
            if (authHeader != null && authHeader.startsWith("Bearer ")) {
                authHeader = authHeader.substring(7);}
            try {jwtUtil.validateToken(authHeader);}
            catch (Exception e) {
                throw new RuntimeException("unauthorized access to application");}}
            return chain.filter(exchange);});
    }

    public static class Config { }
}
```


api-gateway service

```
// filter/AuthenticationFilter.java
...
@Override
pub // filter/RouteValidator.java
r ...
// @Component
i public class RouteValidator {
i     public static final List<String> openApiEndpoints = List.of(
        "api/auth/authenticate",
Str        "api/auth/signup",
exc        "api/auth/login"
        );

    public Predicate<ServerHttpRequest> isSecured =
        request -> openApiEndpoints
            .stream()
            .noneMatch(uri -> request.getURI().getPath().contains(uri));
}
}
```

api-gateway service

```
// filter/AuthenticationFilter.java
...
@Override
pub // filter/RouteValidator.java
r ...
// @Co // filter/RouteValidator.java
i pub ...
i @Component
public class JwtUtil {

    public static final String SECRET =
"4D6351665468576D5A7134743777217A25432A462D4A614E645267556B586E32";
    public void validateToken(final String token) {
        Jwts.parserBuilder().setSigningKey(getSignKey()).build().parseClaimsJws(to
pub ken);
    }
    private Key getSignKey() {
        byte[] keyBytes = Decoders.BASE64.decode(SECRET);
        return Keys.hmacShaKeyFor(keyBytes);
    }
}
```

api-gateway service - application.properties

```
// application.properties
...
##### You need these properties to avoid getting the cors error
spring.cloud.gateway.default-filters=DedupeResponseHeader=Access-Control-Allow-
Credentials Access-Control-Allow-Origin
spring.cloud.gateway.globalcors.corsConfigurations.[/**].allowedOrigins=*
spring.cloud.gateway.globalcors.corsConfigurations.[/**].allowedMethods=*
spring.cloud.gateway.globalcors.corsConfigurations.[/**].allowedHeaders=*
...
```

api-gateway service - application.properties

```
// application.properties
...
##### You need these properties to avoid getting the cors error
spring.cloud.gateway.default-filters=DedupeResponseHeader=Access-Control-Allow-
Credentials Access-Control-Allow-Origin
spring.cloud.gateway.globalcors.corsConfigurations.[/**].allowedOrigins=*
spring.cloud.gateway.globalcors.corsConfigurations.[/**].allowedMethods=*
spring.cloud.gateway.globalcors.corsConfigurations.[/**].allowedHeaders=*
...
```

```
// application.properties
...
##### product-service #####
spring.cloud.gateway.routes[1].id=product-service
spring.cloud.gateway.routes[1].uri=http://localhost:8082/
spring.cloud.gateway.routes[1].predicates[0]=Path=/api/products/**
## We are applying the filter to the product-service route
spring.cloud.gateway.routes[1].filters[0]= AuthenticationFilter
...
```

Building a “secure” M-based project

frontend

Navigation guard, Auth.js, Login, SignUp

```
// ../router/index.js
```

```
... // ../auth.js
```

```
import login from login.vue
```

```
export { login }
```

```
// SignUp.vue
```

```
<template>
```

```
  <div class="form">
```

```
    <h3>SignUp</h3>
```

```
    <label for="username">User Name</label>
```

```
    <input type="text" name="username" required v-model="username">
```

```
    <label for="password">Password</label>
```

```
    <input type="password" name="password" required v-model="password">
```

```
    <button @click="SignUp" class="SignUp">SignUp</button>
```

```
  </div>
```

```
</template>
```

```
</
```

```
...
```

HomeView.vue

```
// HomeView.vue
<button v-if = "authResult" @click="Logout" class="center">Logout</button>
...
<div class="name" v-if="checkRoles('USER')">
  <div class="container">
    <table>
      <tr>
        <th>Code</th>
        <th>Name</th>
        <th>Cost</th>
        <th>Description</th>
      </tr>
      <tr class="item" v-for="product in products" :key="product.id">
        <td>{{ product.code }} </td>
        <td>{{ product.name }} </td>
        <td> {{product.price }}</td>
        <td>{{ product.description }} </td>
      </tr>
    </table>
  </div>
</div>
```

HomeView.vue

```
// HomeView.vue
<button v-if = "authResult" @click="Logout" class="center">Logout</button>
<div class="name" v-if="checkRoles('ADMIN')">
  <div class="container">
    <table>
      ...
      <td><input name="id" type="text" id="id" required v-model=
        "product.id"></td>
      ...
      <td><button class="update" @click="updateProduct(product.id,
        product)">update</button> </td>
      <td><button class="delete" @click="deleteProduct(product.id)">
        delete</button> </td>
    </table>
  </div>
  <h3>Add a Product</h3>
  ...
  <button class="add" @click="addProduct(Aproduct)"> Add </button>
</div>
</div>
```


HomeView.vue - data

```
// HomeView.vue
...
data: function() {
  return {
    authResult: auth.authenticated(),
    products: [],
    Aproduct: {
      id: "",
      code: "",
      name: "",
      price: 0,
      description: "",
    },
    token: localStorage.getItem('jwtToken')
  }
},
```

HomeView.vue – mount()

```
// HomeView.vue
...
mounted() {
  if (this.checkRoles('ADMIN')) {
    const token = localStorage.getItem('jwtToken');
    // Set the Authorization header with the token value
    const headers = {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${token}`
    };
    fetch(`http://localhost:8080/api/products`, { headers })
      .then((response) => response.json())
      .then((data) => (this.products = data))
      .catch((err) => console.log(err.message));
  }
  ...
}
```

HomeView.vue – addProduct(product)

```
// HomeView.vue
...
addProduct(product) {
  fetch(`http://localhost:8080/api/products`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${this.token}`
    },
    body: JSON.stringify(product),
  })
  .then(() => {
    this.$router.push("/")
    window.location.reload();
  })
  .catch((e) => {
    console.log(e);
  });
},
...

```

HomeView.vue – addProduct(product)

```
// HomeView.vue
```

```
...
```

```
addProduct(product) {  
  fetch(`http://localhost:8080/api/products`, {  
    method: "POST",  
    headers: {  
      "Content-Type": "application/json",
```

```
// HomeView.vue
```

```
<button v-if = "authResult" @click="Logout" class="center">Logout</button>  
<div class="name" v-if="checkRoles('ADMIN')">  
  <div class="container">  
    <table>  
      <td><input name="id" type="text" id="id" placeholder="id" required  
        v-model="Aproduct.id"></td>  
      ...  
    <h3>Add a Product</h3>  
    ...  
    <button class="add" @click="addProduct(Aproduct)"> Add </button>  
  </div>  
</div>
```

HomeView.vue – addProduct(product)

```
// HomeView.vue
...
addProduct(product) {
  fetch(`http://localhost:8080/ap
    method: "POST",
    headers: {
      "Content-Type": "applicatio
```

```
// HomeView.vue
<button v-if = "authResult" @cli
<div class="name" v-if="checkRol
  <div class="container">
    <table>
      <td><input name="id" type
        v-model="Aproduct.id"></t
    ...
    <h3>Add a Product</h3>
    ...
    <button class="add" @click="addProduct(Aproduct)"> Add </button>
  </div>
</div>
```

```
// HomeView.vue
...
data: function() {
  return {
    authResult: auth.authenticated(),
    products: [],
    Aproduct: {
      id: "",
      code: "",
      name: "",
      price: 0,
      description: "",
    },
    token: localStorage.getItem('jwtToken')
  },
},
```



Exam dates



Exam dates

Exam: 50 points

Exam dates and duration The duration of the exam is **2 hours and 30 minutes**, and the dates are:

Exam 1: 26.05.2025 - 14:00-16:30 (2.5 hours)

Exam 2: 9.06.2025 - 14:00-16:30 (2.5 hours)

Resit Exam: 20.06.2025 - 14:00-16:30 (2.5 hours)



Final lecture - Course Wrap-up

Course Wrap-up, review, and exam method description

19/5/2025 -> 12/5/2025



UNIVERSITY OF TARTU

**Thank You
for your attention**

Mohamad Gharib
mohamad.gharib@ut.ee



unitartu



tartuuniversity

